EXPERIMENT-60(HDOC)

C Programming

Program: Two Sum Using Pointers and a Nested-Loop Approach

Step 1: Understand the Problem Statement

- Input: The size n and the elements of a variable-length integer array nums[] (positive integers only), and an integer target.
- Output: Two distinct indices i and j (separated by a space) such that nums[i] + nums[j] = target; the indices may be in any order. If no such pair exists, display -1 -1.
- The array is read in main() and passed to `two_sum(int *nums, int n, int target)` using its address; inside the function the elements are accessed only through the pointer (*(nums + i)). The function returns nothing and prints the result itself.
- A nested-loop approach is used: the outer loop picks the first element (index i) and the inner loop picks every later element (index j = i + 1 onwards), so i and j are always distinct and every pair is checked exactly once.
- At most one valid solution is assumed, so the function stops (returns) as soon as the first matching pair is found.

Rule(s) used: A pair (i, j) is valid if i != j and nums[i] + nums[j] = target. If all pairs are checked and none matches, the answer is -1 -1.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Array: 2 7 11 15` `Target: 9`	0 1	0 1	**Pass**
2	`Array: 3 2 4` `Target: 6`	1 2	1 2	**Pass**
3	`Array: 3 3` `Target: 6`	0 1	0 1	**Pass**
4	`Array: 1 2 3` `Target: 7`	-1 -1	-1 -1	**Pass**
5	`Array: 5` `Target: 10`	-1 -1	-1 -1	**Pass**

Step 3: Algorithm

1. Start the program.
2. Read the size n, declare the array nums[n], read its n elements and read the integer target.
3. Call two_sum(nums, n, target), passing the address of the array.
4. In two_sum(), repeat for i = 0 to n - 2 (outer loop).
5. For each i, repeat for j = i + 1 to n - 1 (inner loop).
6. If *(nums + i) + *(nums + j) equals target, print i and j separated by a space and return.
7. If both loops finish without a match, print -1 -1.
8. Stop the program.

Evaluation against Step 2 test cases: tracing the algorithm for [2, 7, 11, 15] with target 9 gives 2 + 7 = 9 at i = 0, j = 1, so the output is "0 1"; for [3, 2, 4] with target 6, the pair (0,1) gives 5 and (0,2) gives 7, then 2 + 4 = 6 at (1,2), so "1 2"; for [3, 3] the only pair (0,1) gives 6, so "0 1"; for [1, 2, 3] with target 7 the largest possible sum is 5, so "-1 -1"; for the single-element array [5] the inner loop never runs, so "-1 -1" — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```c
/* HDOC Day-61: Two Sum using pointers and a nested-loop approach.

   The size n and the elements of a variable-length array nums[] (positive
   integers only) and an integer target are read in main(). The array is
   passed to two_sum() using its address (pointer), and the function accesses
   the elements only through that pointer.

   two_sum() uses two nested loops to find two DISTINCT indices i and j such
   that nums[i] + nums[j] = target, and displays them separated by a space.
   If no such pair exists it displays -1 -1.
*/

#include <stdio.h>

/* Function prototype */
void two_sum(int *nums, int n, int target);

int main() {
    int n, target, i;

    printf("Enter size of array: ");
    scanf("%d", &n);

    int nums[n];                        /* variable-length array */

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    printf("Enter target: ");
    scanf("%d", &target);

    /* pass the address of the array to the function */
    two_sum(nums, n, target);
```

```
    return 0;
}

/* Finds two distinct indices whose elements add up to target */
void two_sum(int *nums, int n, int target) {
    int i, j;

    for (i = 0; i < n - 1; i++) {              /* first element  */
        for (j = i + 1; j < n; j++) {          /* second element */
            if (*(nums + i) + *(nums + j) == target) {
                printf("%d %d\n", i, j);
                return;                        /* at most one solution */
            }
        }
    }

    printf("-1 -1\n");                         /* no valid pair found */
}
```

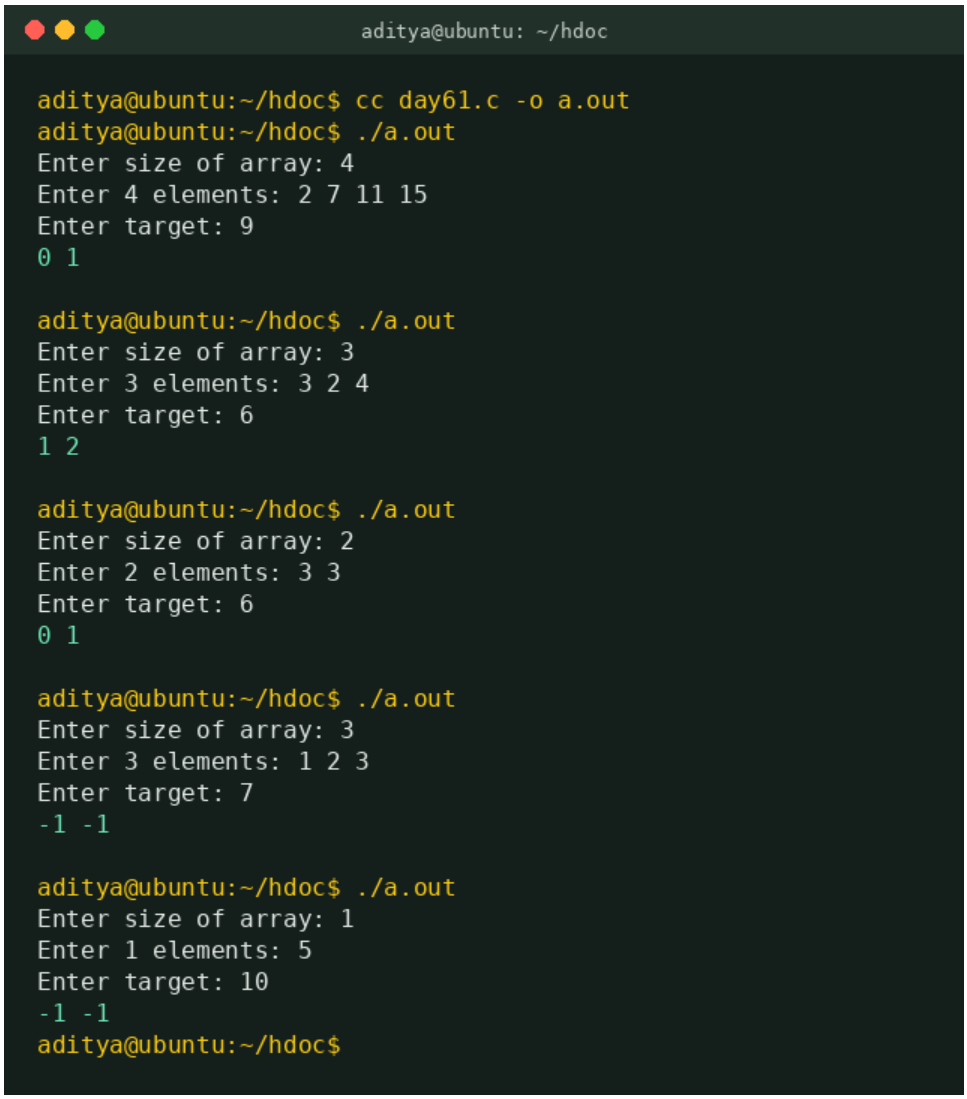
Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

cc day61.c -o a.out	(compile)
./a.out	(execute)

All 5 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot


aditya@ubuntu: ~/hdoc
aditya@ubuntu:~/hdoc$ cc day61.c -o a.out
aditya@ubuntu:~/hdoc$ ./a.out
Enter size of array: 4
Enter 4 elements: 2 7 11 15
Enter target: 9
0 1
aditya@ubuntu:~/hdoc$ ./a.out
Enter size of array: 3
Enter 3 elements: 3 2 4
Enter target: 6
1 2
aditya@ubuntu:~/hdoc$ ./a.out
Enter size of array: 2
Enter 2 elements: 3 3
Enter target: 6
0 1
aditya@ubuntu:~/hdoc$ ./a.out
Enter size of array: 3
Enter 3 elements: 1 2 3
Enter target: 7
-1 -1
aditya@ubuntu:~/hdoc$ ./a.out
Enter size of array: 1
Enter 1 elements: 5
Enter target: 10
-1 -1
aditya@ubuntu:~/hdoc$